PROJECT CASE STUDY

# Upwork Cowork Agent

*An AI-assisted operations dashboard connecting Upwork job discovery, explainable scoring, and portfolio-grounded proposals through an explicit human review workflow.*

---

SYSTEM

**Internal Operations Dashboard**

SCOPE

**Scout · Writer · Critic**
**Proposal History**

APPROACH

**Human-Gated AI**
**Durable Workflows**

## 01 The Problem

Finding suitable freelance work requires more than matching keywords. Operators must assess budgets, skills, client history, competition, and eligibility before deciding which opportunities deserve a tailored proposal.

Disconnected searches, AI drafts, and review notes make that decision difficult to trace. Generating proposals for every shortlisted job also consumes model spend before an operator has chosen to pursue the work.

> Upwork Cowork Agent connects discovery, qualification, drafting, and review in one workspace — automating preparation while keeping generation and approval deliberate human decisions.

## 02 What Was Built

A connected application spanning job evaluation, AI-assisted proposal preparation, review history, and workspace-level configuration.

### JOB DISCOVERY & QUALIFICATION

Find jobs through Upwork MCP, score their fit, and apply competition and location policies. Keep the shortlist separate from the decision to start writing.

### PORTFOLIO-GROUNDED WRITER

Send a selected job to Writer with relevant project evidence. Generate an editable cover letter, screening answers, and bid guidance through the assigned provider.

### CRITIC, REVISION & HISTORY

Request an independent critique, revise deliberately, and preserve proposal versions, review findings, and operator remarks before approval.

### WORKSPACE CONTROL CENTER

Manage scoring preferences, provider credentials, Writer/Critic assignments, and the Upwork OAuth connection behind account-scoped access.

# 03 How It Works

React communicates with a FastAPI API backed by PostgreSQL. Scheduled discovery and explicitly requested generation execute as durable worker jobs, with external integrations kept behind service boundaries.

**Upwork MCP → Scout → Scored shortlist**

Operator selects job → Writer → Optional Critic / revision → Human approval

PostgreSQL → Job snapshots · Proposal versions · Reviews · Durable run history

### Discovery with an explainable policy

Profile-adaptive smart and skill searches feed Scout's manual runs or enabled two-hour schedule. Revisioned workspace rules control fit and quality decisions; source provenance and funnel records make qualification inspectable.

### Explicit generation, grounded in evidence

An eligible job reaches Writer only through an operator action. Writer assembles job context and matched portfolio evidence; Writer and Critic resolve their own configured provider and model without silent fallback.

### Durable execution and traceable review

PostgreSQL queue claims, leases, heartbeats, and fencing tokens protect generation work from stale-worker writes. Saved versions and structured reviews keep edits and critique attached to the proposal they concern.

### A clear external-action boundary

Approval records the human decision; it does not submit to Upwork. Submission-attempt bookkeeping exists, but live submission is disabled by default and the external submission implementation remains outside the current surface.

## 04 Key Features

**Deterministic, explainable scoring** — skills match, historical client spend, and client history feed a configurable rubric, initially weighted 50/30/20. Component scores remain visible.

**Quality-aware shortlisting** — competition thresholds, location rules, and explicit quality states distinguish attractive opportunities from unsuitable or uncertain jobs.

**Deliberate AI spend** — qualifying a job does not trigger Writer. Drafting, critique, and revision each require an explicit request.

**Independent Writer and Critic** — separate provider/model assignments support cross-model review across Anthropic, OpenAI, Gemini, xAI, and Groq integrations.

**Portfolio-backed proposals** — relevant project records and links give Writer concrete evidence to connect past work to the selected job.

**Version-bound critique and revision** — structured rubric scores determine PASS or REVISE. Explicit revisions are capped at two rounds; edits, reviews, and selected remarks stay linked to saved versions.

**Settings-owned Upwork connection** — OAuth with PKCE, encrypted token storage, and refresh belong to the backend rather than a borrowed interactive agent session.

**Account-scoped workspace access** — account-bound sessions, CSRF protection, workspace filtering, and encrypted provider credentials define the application’s access boundaries.

**Connects-aware review** — an on-demand preview exposes application cost, wallet balance, and boost guidance. Operators can record a chosen boost separately from the dollar bid.

**Operational visibility** — durable Scout and generation records expose progress and failures instead of treating long-running work as a browser-only interaction.

# 05 Technical Approach

A modular monolith separates the operator interface, authenticated commands, durable work, and external providers while using PostgreSQL as the shared system of record.

| LAYER | STACK | PURPOSE |
| --- | --- | --- |
| Operator interface | React 19; TypeScript; Vite; TanStack Query; Tailwind CSS 4 | Typed API interaction and server-state-driven dashboard workflows |
| API & access | Python 3.12; FastAPI; Pydantic; session cookies; Argon2 | Validated commands, account authentication, and workspace boundaries |
| Persistence | PostgreSQL; SQLAlchemy async; asyncpg; Alembic | Jobs, proposals, versions, settings, audit records, and durable queues |
| Background work | APScheduler 3.x; Python workers; database leases | Scheduled Scout runs and explicit Writer/Critic execution |
| AI & Upwork | Provider adapters; official MCP SDK; OAuth / PKCE; encrypted vault | Independent model roles and backend-owned Upwork tool access |
| Delivery & checks | Docker; Render configuration; Vercel SPA configuration; pytest; Vitest | Backend packaging, frontend hosting setup, and existing test tooling |

### Design decisions that matter

**Shortlist first, generate on request.** Ranking identifies opportunities without committing the operator to AI calls for every matching job.

**One durable database, explicit workers.** PostgreSQL holds both product history and queued work, avoiding a separate broker for this workload.

**Separate providers from workflow.** Role assignments and normalized service interfaces keep provider-specific logic out of API routes and operator decisions.

## 06 Value Delivered

From disconnected job searches and drafts to a deliberate, inspectable proposal-preparation workflow:

| Focused | Grounded | Traceable |
| --- | --- | --- |
| OPPORTUNITY SELECTION | PROPOSAL PREPARATION | REVIEW & DECISIONS |

- ✓ **A clearer decision on what to pursue.** — Fit scores and discovery-quality signals bring skills, historical client spending, client context, and competition into a structured shortlist.
- ✓ **Preparation without automatic commitment.** — Operators choose where Writer and Critic are used, keeping model spend tied to intentional opportunities rather than every search result.
- ✓ **Evidence that stays with the proposal.** — Project links, saved versions, review findings, and remarks make it possible to inspect how a draft was produced and changed.
- ✓ **Decisions that remain inspectable.** — Approval binds a saved proposal version, while audit history records operator actions. The team can distinguish the reviewed draft from later changes or generation attempts.

*Upwork Cowork Agent — a workspace-scoped application combining explainable discovery, portfolio-grounded AI writing, and durable human review in one connected workflow.*